

Documentación de Funcionalidades: E-commerce Meche Artesanías (Backend Django)

Este documento describe las características principales y la estructura del proyecto de e-commerce "Meche Artesanías", desarrollado con Django para el backend.

1. Estructura General del Proyecto

El proyecto está organizado en una aplicación Django principal

(`meche_artesantias_backend`) y dos aplicaciones específicas: productos y usuarios.

- **meche_artesantias_backend:** Contiene la configuración central del proyecto (`settings.py`, `urls.py`, `wsgi.py`, `asgi.py`). Define la configuración de la base de datos, los middlewares, las plantillas y los archivos estáticos. También gestiona las URL principales del sitio.
- **productos:** Modela y gestiona todo lo relacionado con los productos, categorías, métodos de pago y el carrito de compras (`models.py`, `views.py`, `urls.py`, `serializers.py`).
- **usuarios:** Maneja la autenticación y gestión de usuarios (`models.py`, `views.py`, `urls.py`, `serializers.py`). Utiliza un modelo de usuario personalizado.
- **templates/Tienda:** Contiene los archivos HTML que renderizan las diferentes vistas para el usuario final (páginas de bienvenida, tienda, carrito, login, registro, etc.).
- **static/:** Almacena los archivos estáticos como CSS (Estilos-Css) y JavaScript (Archivos-Js) que dan estilo y funcionalidad interactiva al frontend.

2. Funcionalidades Principales

2.1. Gestión de Usuarios (usuarios app)

- **Modelo de Usuario Personalizado:** Se define un `UsuarioPersonalizado` que hereda del `AbstractUser` de Django, permitiendo futuras personalizaciones. Se utiliza el email como campo principal para el login (`USERNAME_FIELD = 'email'`).
- **Registro de Clientes:**
 - Interfaz de registro (`registro_cliente.html`, `registro-cliente.css`).
 - Validación y envío de datos mediante JavaScript (`registro-cliente.js`).
 - Vistas y lógica en el backend para crear nuevas cuentas de usuario (definidas en `views.py` y enrutadas a través de `urls.py`).
- **Autenticación (Login/Logout):**
 - Página de inicio de sesión (`login.html`, `estilo-login.css`).
 - Lógica de inicio de sesión manejada por JavaScript (`script-login.js`) y vistas de Django.
 - Verificación de autenticación en el lado del cliente (`verificar-autenticacion.js`).
 - El backend gestiona las sesiones de usuario.
- **Administración de Clientes:** Existe una sección para la administración de clientes

2.2. Gestión de Productos y Tienda (productos app)

- **Modelos de Datos:**

- Categoría: Para organizar los productos.
- Producto: Define los artículos en venta con nombre, descripción, precio, stock, categoría e imagen.
- MetodoPago: Define los métodos de pago disponibles.
- Carrito: Representa el carrito de compras asociado a un usuario.
- ElementoCarrito: Representa un producto específico dentro de un carrito con su cantidad.

- **Visualización de Productos:**

- Página de la tienda (tienda.html, Estilos-Tienda.css) que muestra los productos disponibles.
- Lógica en views.py para obtener y mostrar los productos
- Interacción manejada por tienda.js para añadir productos al carrito.

- **Carrito de Compras:**

- Página del carrito (carrito.html, carrito.css) donde los usuarios ven los productos seleccionados.
- Funcionalidad para agregar, ver y posiblemente modificar/eliminar productos del carrito, gestionada por carrito.js y las vistas correspondientes en views.py.
- Los datos del carrito se asocian al usuario autenticado a través de los modelos Carrito y ElementoCarrito.

- **API de Productos:** Se han definido serializers (serializers.py) y rutas de API (urls.py) usando Django REST Framework, lo que sugiere que hay endpoints para listar y ver detalles de productos y categorías mediante API. Esto permite que el frontend (u otras aplicaciones) interactúe con los datos de productos de forma estructurada.

2.3. Interfaz de Usuario y Frontend

- **Páginas Estáticas/Informativas:**
 - Página de Bienvenida (pgbienvenida.html, Estilos-pgbienvenida.css, script-pgbienvenida.js).
 - Página "Nosotros" (nosotros.html, Estilos-Nosotros.css).
- **Dashboard:** Existe una sección de Dashboard (dashboard.html, dashboard.js, varios CSS). una interfaz para usuarios registrados para gestionar pedidos, perfil y productos
- **Estilos y Scripts:** Se utiliza CSS para dar estilo a todas las páginas y JavaScript para añadir interactividad, como la gestión del carrito, validaciones de formularios y la carga dinámica de contenido en el dashboard.

3. Administración (Django Admin)

- Los modelos Categoría, Producto, MetodoPago, Carrito, ElementoCarrito y UsuarioPersonalizado están registrados en el sitio de administración de Django (admin.py en productos y usuarios). Esto proporciona una interfaz web automática para que los administradores gestionen usuarios, productos, categorías y otros datos directamente.

4. Resumen de Funcionalidades Clave

- Registro e inicio de sesión de usuarios.
- Visualización de catálogo de productos por categorías.
- Carrito de compras
- Gestión de productos, categorías y usuarios a través del panel de administración de Django.
- API REST para acceder a los datos de productos y categorías.
- Interfaz de usuario frontend con páginas de bienvenida, tienda, carrito, login/registro y nosotros.